

A natural presentation/viva script covering the complete ML workflow without going deeply into mathematical theory.

1. Introduction

Hello everyone.

Today I am going to explain my machine learning project, **DiabeticLens — Explainable Machine Learning for Diabetes Prediction**.

The main idea behind this project is to use machine learning to predict whether a patient is likely to belong to the **diabetes or non-diabetes class**, based on different health and lifestyle features.

I focused mainly on the **machine learning part**, starting from understanding the dataset, preprocessing the data, training different models, evaluating them, tuning the final model, and finally integrating the model into a Streamlit application.

2. Dataset

So first, let's talk about the dataset.

My dataset contains **100,000 patient records and 9 columns**.

The features are:

Feature	Type	Role
gender	Categorical	Input
age	Numerical	Input
hypertension	Binary	Input
heart_disease	Binary	Input
smoking_history	Categorical	Input
bmi	Numerical	Input
HbA1c_level	Numerical	Input
blood_glucose_level	Numerical	Input
diabetes	Binary	Target

0 → No Diabetes

1 → Diabetes

3. Understanding the Data

Before directly training a model, I first performed **data understanding**.

I checked the shape of the dataset, data types, missing values, duplicate records, categorical values and the distribution of the target variable.

There were **100,000 rows**, and there were no missing values.

However, I found around **3,854 duplicate records**.

So I removed those duplicate records.

After removing duplicates, I had **96,146 records**.

I also checked the target distribution. Approximately **8.5% of the records belong to the diabetes class**, while the majority belong to the non-diabetes class.

So this is an **imbalanced classification problem**.

Because of this, I knew that I should not judge my model only using accuracy.

4. Exploratory Data Analysis

After understanding the dataset, I performed **Exploratory Data Analysis, or EDA**.

I looked at the distributions of different features using graphs such as histograms and boxplots.

I also checked the relationship between features and the diabetes target.

For example, I analyzed features such as:

- Age
- BMI
- HbA1c level
- Blood glucose level
- Hypertension
- Heart disease
- Smoking history

5. Data Preprocessing

After EDA, the next step was preprocessing.

I separated my dataset into **X**, which contains the input features, and **y**, which contains the diabetes target.

Then I divided the data into training and testing sets.

I used an **80-20 split**.

So around **80% was used for training** and **20% was used for testing**.

I also used stratification so that the distribution of diabetes and non-diabetes cases was maintained in both sets.

6. Handling Different Feature Types

My dataset contains different types of features, so I could not apply exactly the same preprocessing to everything.

For numerical features such as age, BMI, HbA1c and blood glucose, I used **StandardScaler**.

For categorical features such as gender and smoking history, I used **OneHotEncoder**.

And binary features such as hypertension and heart disease were passed through directly.

I combined these transformations using a **ColumnTransformer**.

This preprocessing pipeline was then saved using Joblib so that I could use the exact same preprocessing when making predictions in my application.

7. Model Training

After preprocessing, I moved to the model training stage.

Instead of using only one algorithm, I trained multiple machine learning classification models and compared their performance.

The models I tested were:

- Logistic Regression
- Decision Tree
- Random Forest
- K-Nearest Neighbors
- Gradient Boosting

8. Model Evaluation

For evaluation, I used multiple metrics.

The main metrics were:

This was important because my dataset was imbalanced.

For example, accuracy tells me how many predictions were correct overall.

But recall tells me how many actual diabetes cases the model was able to identify.

And F1-score gives a balance between precision and recall.

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

9. Initial Model Results

After training the models, I got the following results.

For **Logistic Regression**, the accuracy was around **95.97%**, with an F1-score of **73.65%**.

For **Decision Tree**, the accuracy was around **94.84%**, and its recall was about **74.12%**.

For **Random Forest**, the accuracy was around **96.93%**, with an F1-score of **79.84%**.

For **KNN**, the accuracy was around **95.98%**, with an F1-score of **73.07%**.

And for **Gradient Boosting**, I got around **97.17% accuracy** and an **81.10% F1-score**.

So Gradient Boosting showed strong overall performance and I selected it for further evaluation and tuning.

10. Model Evaluation and Tuning

After the initial comparison, I didn't immediately consider the model finished.

I performed further evaluation using:

I also used **5-fold Stratified Cross-Validation** to check the model's performance across different splits of the training data.

After that, I performed **hyperparameter tuning** using RandomizedSearchCV.

The main objective during tuning was to improve the **F1-score**, because the dataset is imbalanced and I wanted to consider both precision and recall.

- Confusion matrices
- ROC curves
- Precision-Recall curves
- Cross-validation

11. Threshold Tuning

Another important part of my project was **decision threshold tuning**.

Normally, a binary classification model can use a probability threshold of 0.5.

But instead of directly assuming 0.5 was the best threshold, I tested different thresholds from **0.10 to 0.90**.

I compared precision, recall and F1-score at different thresholds.

Based on this analysis, the best F1-score was obtained at a threshold of **0.45**.

Therefore, my final prediction logic became:

```
prediction = int(probability >= 0.45)
```

Probability < 0.45 → No Diabetes

Probability >= 0.45 → Diabetes

12. Final Model

After tuning, my final model was the **Tuned Gradient Boosting model**.

The final model achieved the following results on the test set:

Metric	Final Test Score
Accuracy	97.18%
Precision	98.82%
Recall	68.87%

F1 Score	81.17%
ROC-AUC	97.75%
PR-AUC	88.42%

13. Saving the Model

After completing the training and evaluation, I saved the final model using Joblib.

The main files are:

The reason I saved the preprocessor separately is that new user input must go through the **same preprocessing steps used during training**.

And I saved the threshold separately so that the application uses the same decision rule as the evaluation stage.

```
final_diabetes_model.pkl
preprocessor.pkl
decision_threshold.pkl
```

14. Streamlit Application

After completing the machine learning part, I integrated the model into a **Streamlit application**.

The application provides a simple interface where the user can enter:

After entering these values, the application sends the information through the saved preprocessing pipeline.

Then the processed data is passed to the final Gradient Boosting model.

The model produces a probability.

- Gender
- Age
- BMI
- HbA1c level
- Blood glucose level
- Smoking history
- Hypertension
- Heart disease

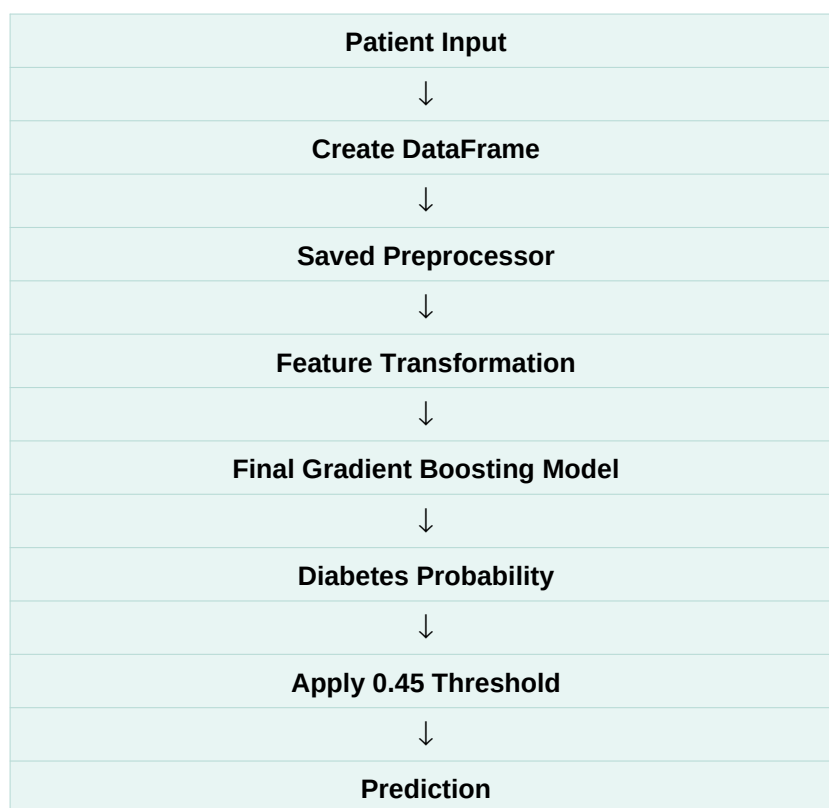
15. Prediction Output

For example, if the probability is **3.60%**, then it is below the threshold of 45%, so the application displays:

Model Prediction: No Diabetes.

16. Complete Application Flow

So the complete flow of my project is:



17. Future Scope

For future improvement, I want to add **Explainable AI using SHAP**.

The purpose of SHAP would be to explain **why the model made a particular prediction**.

For example, instead of only saying that the diabetes probability is 70%, the system could also show which features contributed more toward that prediction.

I also plan to improve the dashboard, add better visual explanations and potentially deploy the application online.

18. Limitations

There are also some limitations to my project.

First, the model is trained on a particular dataset, so its performance may not be the same on every real-world population.

Second, the dataset is imbalanced, with fewer diabetes cases than non-diabetes cases.

And most importantly, this project is an **ML prediction or screening project**, not a medical diagnostic system.

The predictions should not replace professional medical evaluation.

19. Conclusion

So, to summarize my project:

I started with a dataset containing **100,000 patient records**.

I performed data understanding and EDA, removed duplicate records, and ended up with **96,146 records**.

Then I performed preprocessing using **StandardScaler** and **OneHotEncoder**.

I trained and compared multiple machine learning algorithms.

After evaluation, cross-validation and hyperparameter tuning, I developed a **Tuned Gradient Boosting model**.

I also tuned the classification threshold and selected **0.45 based on F1-score analysis**.

The final model achieved **97.18% accuracy, 98.82% precision, 68.87% recall, 81.17% F1-score, 97.75% ROC-AUC, and 88.42% PR-AUC**.

Finally, I integrated the model into a **Streamlit application** where users can enter patient information and receive a diabetes probability and prediction.

That is the complete workflow of my **DiabeticLens project**.

Thank you.

20. Model Comparison at a Glance

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	95.97%	86.99%	63.86%	73.65%	95.99%
Decision Tree	94.84%	69.45%	74.12%	71.71%	85.58%
Random Forest	96.93%	94.96%	68.87%	79.84%	95.92%
KNN	95.98%	89.13%	61.91%	73.07%	90.04%
Gradient Boosting	97.17%	98.73%	68.81%	81.10%	97.74%

Final model: Tuned Gradient Boosting

Decision threshold: 0.45

The 0.45 value is the project's machine-learning decision threshold, not a clinical diagnostic cutoff.